

---

# My SQL

By Priti Dalvi

# Index

---

| Module      | Topic                                | Page No |
|-------------|--------------------------------------|---------|
| Module 1 :  | Getting Started with MySQL           | 4       |
| Module 2 :  | Editing SQL Commands                 | 17      |
| Module 3 :  | Data Retrieval & Ordering Output     | 25      |
| Module 4 :  | Regular Expressions                  | 45      |
| Module 5 :  | Creating Tables                      | 48      |
| Module 6:   | Inserting, Modifying & Deleting Data | 57      |
| Module 7 :  | Modifying Table Structure            | 65      |
| Module 8 :  | Integrity Constraints                | 71      |
| Module 9 :  | Built-In Functions                   | 86      |
| Module 10 : | Indexes                              | 99      |
| Module 11 : | Advanced Queries                     | 105     |

# Index

---

| Module      | Topic                          | Page No |
|-------------|--------------------------------|---------|
| Module 12 : | Views                          | 119     |
| Module 13 : | Transaction Processing         | 127     |
| Module 14 : | Miscellaneous function         | 133     |
| Module 15:  | Introduction to programming    | 139     |
| Module 16 : | Temporary tables               | 151     |
| Module 17 : | Cursors                        | 155     |
| Module 18 : | Stored Procedure and Functions | 169     |
| Module 19 : | Triggers                       | 185     |
| Module 20:  | MySQL Prepared Statement       | 200     |
| Module 21:  | Exception Handling             | 208     |

# Module 1. Getting Started with MySQL

---

- Overview
  - Introduction to Databases
  - Introducing MySQL
  - Main Components of MySQL
  - Starting MySQL commands
  - Exiting MySQL

# Introduction to Databases

---

Computerized record-keeping system.



## *EMPLOYEE*

| EMPNO | ENAME  | JOB      | MANAGER | HIREDATE    | SALARY | COMMISS<br>ON | DEPTNO |
|-------|--------|----------|---------|-------------|--------|---------------|--------|
| 7369  | SMITH  | CLERK    | 7902    | 17-DEC-1980 | 800    |               | 20     |
| 7499  | ALLEN  | SALESMAN | 7698    | 20-FEB-1981 | 1600   | 300           | 30     |
| 7521  | WARD   | SALESMAN | 7698    | 22-FEB-1981 | 1250   | 500           | 30     |
| 7566  | JONES  | MANAGER  | 7839    | 02-APR-1981 | 2975   |               | 20     |
| 7654  | MARTIN | SALESMAN | 7698    | 28-SEP-1981 | 1250   | 1400          | 30     |
| 7698  | BLAKE  | MANAGER  | 7839    | 01-MAY-1981 | 2850   |               | 30     |
| 7782  | CLARK  | MANAGER  | 7839    | 09-JUN-1981 | 2450   | 0             | 10     |
| 7788  | SCOTT  | ANALYST  | 7566    | 19-APR-1987 | 3000   |               | 20     |

# Introducing SQL

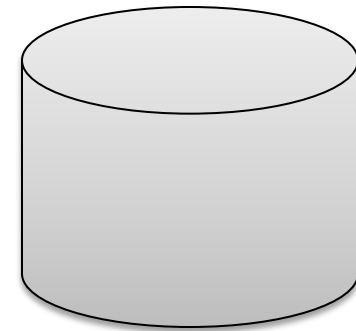
---

**SQL Statement  
Is entered**

```
mysql> SELECT LOC FROM DEPT;
```

**Statement is sent  
To the Database**

**Database**



**Data is displayed**

```
+----+  
| LOC |  
+----+  
| NEW YORK |  
| DALLAS   |  
| CHICAGO  |  
| BOSTON   |  
+----+  
4 rows in set (0.00 sec)  
mysql>
```

# What is MySQL?

---

- MySQL is an **open source relational database**.
- MySQL is **cross platform**
- MySQL supports **multiple storage engines**
- MySQL has **high performance**
- **Cost effective**

# Why MySQL?

---

- MySQL database has become the world's most popular **open source database**.
- It has high performance, high reliability, and easy to use.
- It runs on more than 20 platforms.
- It offers comprehensive range of database tools.
- Source code is available under the terms of GNU General public Licenses.

# The application which uses MySQL

---

- The application which uses MySQL includes:
  - TYPO3
  - Joomla
  - PHP
  - Drupal
  - Wordpress
  - Other Softwares built on LAMP stack.

# Features

---

- Web applications
- Open-source support
- Available large table size
- Stability

# Comparison

---

## MySQL Comparison to other RDBMS Products

| <i>Feature</i>        | <i>MySQL</i> | <i>Oracle</i> | <i>MS SQL Server</i> | <i>PostgreSQL</i> |
|-----------------------|--------------|---------------|----------------------|-------------------|
| Transactional         | Yes          | Yes           | Yes                  | Yes               |
| Open-source           | Yes          | No            | No                   | Yes               |
| TCO                   | Low          | High          | High                 | Low               |
| Development languages | Many         | Many          | Fewer                | Many              |
| Enterprise user base  | Yes          | Yes           | Yes                  | No                |
| Company support       | Yes          | Yes           | Yes                  | No?               |
| Cross-platform        | Yes          | Yes           | No                   | Yes               |

# Platforms and Interfaces

---

- Many programming languages with language specific APIs includes libraries for accessing MySQL database.
- MySQL connector-net is used for integration with Microsoft Visual Studio.
- MySQL connector-java for Java
- An ODBC interface called MYODBC allows interface with programming languages such as ASP and Coldfusion.

# Main Components of MySQL

---

```
MySQL Command Line Client
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 4
Server version: 5.0.37-community-nt MySQL Community Edition (GPL)

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| priti |
| test |
+-----+
4 rows in set (0.00 sec)

mysql> connect priti;
Connection id: 5
Current database: priti

mysql>
```

# Main Components of MySQL

---

```
MySQL Command Line Client
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 7
Server version: 5.0.37-community-nt MySQL Community Edition (GPL)

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql> connect priti
Connection id: 8
Current database: priti

mysql> show tables;
+-----+
| Tables_in_priti |
+-----+
| dept             |
| emp              |
| emp1             |
+-----+
3 rows in set (0.00 sec)

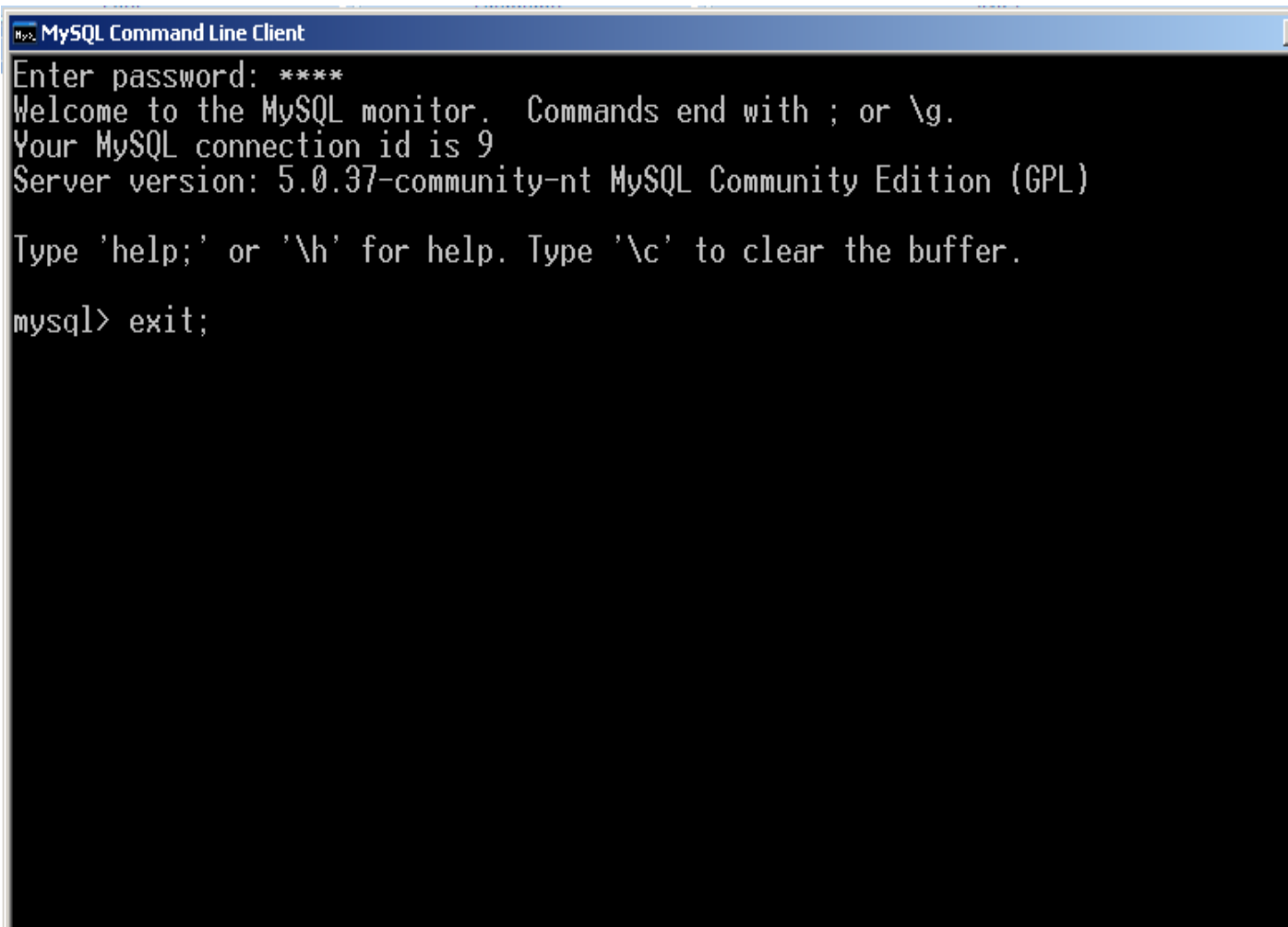
mysql> select * from emp;
+-----+-----+-----+
| empFname | empLname | sal  |
+-----+-----+-----+
| priti    | dalvi    | 10000 |
| nita     | sapte    | 5000  |
+-----+-----+-----+
2 rows in set (0.08 sec)

mysql>
```

# Exiting MySQL

---

mysql> EXIT

A screenshot of a Windows-style application window titled "MySQL Command Line Client". The window has a blue title bar and a black main area with white text. The text inside the window shows the MySQL login sequence: "Enter password: \*\*\*\*", "Welcome to the MySQL monitor. Commands end with ; or \g.", "Your MySQL connection id is 9", and "Server version: 5.0.37-community-nt MySQL Community Edition (GPL)". It also includes instructions: "Type 'help;' or '\h' for help. Type '\c' to clear the buffer." At the bottom, the prompt "mysql> exit;" is shown.

```
MySQL Command Line Client
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 5.0.37-community-nt MySQL Community Edition (GPL)

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql> exit;
```

# MySQL Workbench

---

- Visual tool for database architects, developers, and DBAs
- A free integrated environment developed by MySQL
- Enables users to graphically administer MySQL databases
- Lets users manage database design & modeling
- Database administration

# Module 2. Editing SQL Commands

---

- Overview
  - Entering SQL commands
  - Editing SQL commands
  - Managing SQL files

# Entering and Editing SQL Commands

---

- Terminating SQL statement:
  - a semicolon at the end of a line,
  - a semicolon on a line by itself,
- Viewing details of SQL buffer:
  - `mysql>SHOW ENGINE INNODB STATUS\G;`
- Viewing databases:
  - `mysql>Show databases;`
  - `mysql>Show databases like 'test%';`

# Entering and Editing SQL Commands

---

mysql > Show databases(LIKE <*wildcard*>);

mysql> Show databases;

mysql > Connect databasename;

or

mysql > use databasename

mysql > desc emp;

mysql > desc emp 'e%';

mysql > Show columns from emp;

mysql > Show columns from emp like 'e%';

mysql > Show tables;

mysql > Show tables (from<*databasename*>) (LIKE *wildcard*>);

mysql > Show table status like 'emp';

mysql > Tee c:\a.txt;

# Entering and Editing SQL Commands

---

mysql > SHOW CREATE TABLE emp;

mysql > explain select \* from emp;

mysql > explain select empno, ename from emp;

mysql > check table emp;

# Executing SQL files

---

A file of SQL commands such as books.sql can also be executed from inside the MySQL client using the source command

```
mysql > Source c:\.....\query.sql
```

Here the full path to query.sql should be used.

# Show the Structure of the table

---

Mysql> desc emp;

```
mysql>
mysql>desc emp;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| empFname   | varchar(15)   | YES  |     | NULL    |       |
| empLname   | varchar(15)   | YES  |     | NULL    |       |
| sal        | int(11)       | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>_
```

# Entering and Editing SQL Commands

---

*mysql>help;*

*mysql> \c*

*mysql> Show variables;*

*mysql > Prompt*

*mysql > Prompt Priti*

*mysql > Create database Pragati;*

*mysql > Create database if not exists Pragati;*

*mysql > drop database Pragati;*

*mysql > drop database IF EXISTS Pragati;*

# Entering and Editing SQL Commands

---

```
mysql> Select version(), current_date();
```

```
mysql> select now();
```

```
mysql> select user();
```

# Module 3. Data Retrieval & Ordering Output

---

- Overview

- Simple Data Retrieval
- Describing Table Structure
- Conditional Retrieval using Arithmetic, Relational, Logical and Special Operators
- The ORDER BY clause.
- Aggregate functions
- The GROUP BY and HAVING clause

# Data Retrieval

---

```
SELECT <select phrase> [ FROM <table(s)> [ WHERE  
<where phrase> ]  
[ GROUP BY <group-by clause> ] [ORDER BY <order-by  
clause> ]  
[ LIMIT # of rows ] ]
```

# Data Retrieval

---

```
mysql> SHOW TABLES;
```

```
mysql >SELECT * FROM emp;
```

```
mysql > SELECT empno, ename FROM emp;
```

```
mysql > SELECT empno as 'employee NUM', ename FROM emp;
```

(heading will be displayed in the same case)

```
mysql > SELECT distinct deptno FROM emp;
```

```
mysql> SELECT * FROM emp limit 2;
```

```
mysql > SELECT distinct deptno FROM emp
```

Limit 2;

```
mysql > SELECT distinct dept_code FROM emp
```

Limit 4, 15;

```
mysql > delete from dept limit 2; (first2 records are deleted)
```

# Conditional Retrieval

---

```
mysql> SELECT * FROM employee WHERE salary > 3500;
```

```
mysql> SELECT    emp_code, emp_name FROM employee WHERE  
                dept_code = 'MKTG';
```

Or

```
mysql> SELECT    emp_code, emp_name FROM employee WHERE  
                dept_code = "MKTG";
```

```
mysql> SELECT * FROM employee WHERE hire_date >= '1998-1-1';
```

# Description

---

||                      or operator

:=                      operator-Used to set variables

Binary                Field attribute.. Controls case sensitivity

# Relational Operators

---

|    |                          |
|----|--------------------------|
| =  | equal to                 |
| != | not equal to             |
| <> | not equal to             |
| >  | greater than             |
| <  | less than                |
| >= | greater than or equal to |
| <= | less than or equal to    |

# Relational Operators

---

```
mysql >SELECT * FROM employee WHERE salary > 3000;
```

```
Mysql>SELECT emp_name FROM employee WHERE dept_code !=  
        'MKTG';
```

```
Mysql>SELECT * FROM employee WHERE emp_name = 'Vijay  
        Gupta';
```

```
mysql>SELECT * FROM employee WHERE binary emp_name =  
        'Vijay Gupta';
```

# Logical Operators

---

|         |     |
|---------|-----|
| NOT, !  | not |
| OR,     | or  |
| AND, && | and |

# Logical Operators

---

## The **AND** Operator

```
mysql > SELECT * FROM employee  
        WHERE dept_code = 'MKTG' AND sex = 'F';
```

```
mysql > SELECT * FROM employee  
        WHERE salary >= 3000 AND salary <= 4000;
```

## The **OR** Operator

```
mysql > SELECT * FROM employee  
        WHERE dept_code = 'MKTG' OR dept_code = 'FIN';
```

## The **NOT** Operator

```
mysql > SELECT * FROM employee WHERE NOT dept_code =  
        'MKTG';
```

# Logical Operators

---

```
mysql>select * from emp  
where sal < 6000 || deptno=20;
```

```
mysql>select * from emp  
where sal<6000 && deptno=20;
```

```
mysql>select * from emp  
where not(deptno=20);
```

# Special Operators

---

The **BETWEEN** operator

```
mysql > SELECT * FROM employee WHERE salary BETWEEN 3000  
and 4000;
```

```
mysql > SELECT * FROM employee WHERE date_join BETWEEN  
'2001-01-01' and '2001-12-31'
```

The **IN** operator

```
mysql > SELECT * FROM employee WHERE dept_code IN ('MKTG',  
'FIN');
```

```
mysql > SELECT * FROM employee  
WHERE dept_code NOT IN ('MKTG', 'FIN');
```

# Special Operators

---

The **LIKE** operator

```
mysql> SELECT * FROM employee WHERE emp_name LIKE  
      'P%';
```

```
mysql> SELECT * FROM employee WHERE emp_name LIKE  
      '%Gupta';
```

```
mysql> SELECT * FROM employee WHERE emp_name LIKE  
      '%Gupta%';
```

```
mysql> SELECT * FROM employee WHERE emp_name NOT  
LIKE   '%Gupta%';
```

```
mysql> SELECT * FROM employee WHERE grade LIKE '_1';
```

# Special Operators

---

The **IS NULL** operator

```
mysql> SELECT * FROM employee WHERE reports_to IS NULL;
```

Or

```
mysql> SELECT * FROM employee WHERE reports_to <=> NULL;
```

```
mysql> SELECT * FROM employee WHERE reports_to IS NOT  
      NULL;
```

# Arithmetic Operators

---

|   |                |
|---|----------------|
| + | addition       |
| - | subtraction    |
| * | multiplication |
| / | division       |

```
mysql> SELECT * FROM product  
      WHERE direct_sales + indirect_sales > target;
```

```
mysql> SELECT prod_code, prod_name, direct_sales + indirect_sales  
      FROM product;
```

```
mysql> SELECT prod_code,  
      direct_sales + indirect_sales "Total sales"  
      FROM product;
```

# Ordering the SELECT Query Output

---

## Ordering on single column

```
mysql> SELECT * FROM employee ORDER BY emp_code;
```

```
mysql> SELECT * FROM employee ORDER BY 1;
```

```
mysql> SELECT * FROM employee WHERE sex = 'M' ORDER BY  
emp_name;
```

```
mysql> SELECT * FROM employee ORDER BY age DESC;
```

## Ordering on multiple columns

```
mysql> SELECT * FROM employee ORDER BY dept_code,  
emp_name;
```

```
mysql> SELECT * FROM employee ORDER BY dept_code, age  
DESC;
```

# Aggregate Functions

---

```
mysql> SELECT COUNT (*) FROM employee;
```

```
mysql> SELECT SUM (salary) FROM employee;
```

```
mysql> SELECT AVG (age) FROM employee;
```

```
mysql> SELECT MAX (salary) FROM employee;
```

```
mysql> SELECT MIN (salary) FROM employee;
```

# The GROUP BY clause

---

```
mysql> SELECT dept_code, sum (salary)
        FROM employee
        GROUP BY dept_code;
```

OR

```
mysql> SELECT dept_code, sum (salary)
        FROM employee
        GROUP BY 1;
```

```
Mysql>SELECT *, sum(sal)
        FROM emp
        GROUP BY deptno;
```

# The HAVING Clause

---

```
mysql> SELECT dept_code, sum (salary) FROM employee  
        GROUP BY dept_code  
        HAVING sum (salary) > 10000;
```

```
mysql> SELECT dept_code, sum (salary)  
        FROM employee  
        WHERE age > 30  
        GROUP BY dept_code  
        HAVING sum (salary) > 10000  
        ORDER BY sum (salary) desc;
```

# Counting and Identifying Duplicates

---

```
SELECT COUNT(*) as repetitions, last_name, first_name  
FROM person  
GROUP BY last_name, first_name  
HAVING repetitions > 1;
```

# Eliminating Duplicates

---

```
SELECT last_name, first_name  
FROM person  
GROUP BY last_name, first_name;
```

# Module 4. Regular Expressions

---

- Overview
  - using regular expressions

# Regular expressions

---

- “.” matches any single character
- “\*” matches zero or more

```
mysql>SELECT * FROM employee WHERE name REGEXP '^b';
```

```
mysql>SELECT * FROM employee WHERE name REGEXP '[abc]';
```

```
mysql>SELECT * FROM employee WHERE name REGEXP '[a-c]';
```

```
mysql>SELECT * FROM employee WHERE name REGEXP '^A';
```

```
mysql>SELECT * FROM employee WHERE name REGEXP 'T$';
```

# Regular expressions

---

```
mysql> SELECT * FROM employee WHERE name REGEXP BINARY  
      '^K';
```

```
mysql> SELECT * FROM pet WHERE name REGEXP  
      '^.....$';
```

# Module 5. Creating Tables

---

- Overview
  - Creating a Table
  - Data Types

# Creating Tables

---

```
CREATE [ temporary ] TABLE [ if not exists ] tablename (  
    column-name data-type [other clauses]... );
```

```
mysql> CREATE TABLE dept (  
    dept_code varchar (4),  
    dept_name varchar(20) );
```

# Creating Tables

---

```
mysql> CREATE TABLE dept (  
        dept_code varchar (4),  
        dept_name varchar (20) );
```

```
mysql> CREATE TABLE IF NOT EXISTS dept (  
        dept_code varchar (4),  
        dept_name varchar (20) );
```

# Creating Tables

---

```
Mysql>CREATE TABLE CUSTOMER(  
id int not null auto_increment primary key,  
Name varchar(20),  
City enum('mumbai','delhi') default 'mumbai'  
);
```

# Numeric Data Types:

| Data Type  | Range                                                                                                                            | Stores       |
|------------|----------------------------------------------------------------------------------------------------------------------------------|--------------|
| TINYINT    | If signed -128 to 127, If unsigned from 0 to 255,Storage (Bytes): 1B                                                             | Integer Data |
| SMALL INT  | If signed -32768 to 32767, from 0 to 65535, width up 5 digits, to Storage (Bytes): 2B                                            | Integer Data |
| MEDIUM INT | If signed -8388608 to 8388607, if unsigned from 0 to 16777215, Storage (Bytes): 3B                                               | Integer Data |
| INT        | If signed -2147483648 to 2147483647, If unsigned range is from 0 to 4294967295. width of up to 11 digits.<br>Storage (Bytes): 4B | Integer Data |
| BIGINT     | If signed -9223372036854775808 to 9223372036854775807, if unsigned from 0 to 18446744073709551615, Storage (Bytes): 8B           | Integer Data |

# Numeric Data Types:

---

| Data Type     | Range                                                                                                                  | Stores                |
|---------------|------------------------------------------------------------------------------------------------------------------------|-----------------------|
| FLOAT(M,D)    | A precision from 0 to 23 results in a 4-byte, will default to 10,2. Decimal precision can go to 24 places              | floating-point number |
| DOUBLE(M, D)  | precision from 24 to 53 results in an 8-byte double-precision, default to 16,4. Decimal precision can go to 53 places. | floating-point number |
| DECIMAL(M, D) | NUMERIC is a synonym for DECIMAL                                                                                       | floating-point number |

# Numeric Data Types:

---

| Data Type | Range                                                                               | Stores             |
|-----------|-------------------------------------------------------------------------------------|--------------------|
| DATE      | Between 1000-01-01 and 9999-12-31, YYYY-MM-DD default format                        | Date data          |
| DATETIME  | between 1000-01-01 00:00:00 and 9999-12-31 23:59:59, in YYYY-MM-DD HH:MM:SS format  | Date and Time data |
| TIMESTAMP | DATETIME format, only without the hyphens eg: as 19731230153000 ( YYYYMMDDHHMMSS ). | Date and Time data |
| TIME      | Stores the time in HH:MM:SS format.                                                 |                    |
| YEAR(M)   | Stores a year in 2-digit or 4-digit format, The default length is 4.                |                    |

# String Types:

---

| Data Type                | Range                                                      | Stores                                                                 |
|--------------------------|------------------------------------------------------------|------------------------------------------------------------------------|
| CHAR(M) -                | n characters where n can be 1 to 255                       | Fixed-length character data.                                           |
| VARCHAR(M)               | 1 to 255 characters in length                              | variable-length data                                                   |
| TEXT                     | 1 to 65535 characters, used to store large amounts of data | Store text                                                             |
| BLOB                     | Binary Large Objects                                       | store large amounts of binary data like images or other types of files |
| TINYBLOB or TINYTEXT     | 255 characters                                             |                                                                        |
| MEDIUMBLOB or MEDIUMTEXT | length of 16777215 characters                              |                                                                        |

# String Types:

---

| <b>Data Type</b>      | <b>Range</b>                          | <b>Stores</b> |
|-----------------------|---------------------------------------|---------------|
| LOB<br>or<br>LONGTEXT | length of 4294967295 characters       |               |
| ENUM                  | Enumeration, creating a list of items |               |

# Module 6. Inserting, Modifying & Deleting Data

---

- Overview

- Inserting Data into a Table
- Inserting Data into a Table using Sub query
- Modifying Data in a Table
- Deleting Data from a Table

# Insert records

---

Syntax:

```
INSERT [ low_priority | delayed ] [ ignore ] [ into ]  
<tablename>  
[ ( <columnname>, ... ) ] VALUES (<insert expression> ,)
```

```
INSERT INTO table-name VALUES (value1, value2, ...);
```

```
Insert into customer values(null,'priti','mumbai');
```

```
mysql> INSERT INTO dept VALUES ('MKTG', 'Marketing');
```

```
mysql> INSERT INTO dept VALUES ('FIN', 'Finance');
```

```
mysql> INSERT INTO dept VALUES ('TRNG', 'Training');
```

# Inserting Data into Table

---

```
mysql> INSERT INTO employee (emp_code, age, emp_name)  
VALUES(101, 33, 'Sunil');
```

# Insert records

---

Inserting the records using another table

```
INSERT [LOW_PRIORITY | DELAYED] [IGNORE] [INTO]  
      tbl_name[(col_name,...)] SELECT ...
```

```
insert into emp (empno,ename) select roll, name from t6;
```

```
mysql> INSERT INTO senior  
      SELECT * FROM employee WHERE age > 50;
```

```
mysql> INSERT INTO senior (empno, ename)  
      SELECT empno, ename FROM employee WHERE age > 50;
```

# Inserting Data into a Table

---

```
Mysql> desc dept;
```

| Field     | Type        | Null? | Key   | Default | Extra |
|-----------|-------------|-------|-------|---------|-------|
| -----     | -----       | ----- | ----- | -----   | ----- |
| DEPT_CODE | varchar(4)  | yes   |       | null    |       |
| DEPT_NAME | varchar(20) | yes   |       | null    |       |

# Counting and Identifying Duplicates:

---

```
CREATE TABLE person
(
  first_name CHAR(20) NOT NULL,
  last_name CHAR(20) NOT NULL,
  sex CHAR(10),
  PRIMARY KEY (last_name, first_name)
);
INSERT IGNORE INTO person (last_name, first_name)
VALUES( 'Rampal', 'Arjun');
REPLACE INTO person (last_name, first_name)
VALUES( 'Rampal', 'Arjun');
REPLACE INTO person (last_name, first_name)
VALUES( 'Rampal', 'Arjun');
```

# Modifying and Deleting Data

---

UPDATE *<tablename>* SET *<columnname>* = *<value>* ( ,  
    *<columnname>*  
= *<value>* ) [ WHERE *<where phrase>* ] [ LIMIT *<# of rows>* ];

mysql> **UPDATE** employee **SET** salary = salary + 100;

mysql> **UPDATE** employee **SET** salary = salary + 200 WHERE sex = 'F';

# Modifying and Deleting Data

---

DELETE [LOW PRIORITY or QUICK] FROM *table\_name*  
[WHERE *where\_definition*]  
[ORDER BY *order\_definition*]  
[LIMIT *row\_value*]

mysql> **DELETE** FROM employee;

mysql> **DELETE** FROM employee WHERE dept\_code = 'MKTG';

Mysql> **TRUNCATE** employee;

# Modifying and Deleting Data

---

DELETE [LOW\_PRIORITY | QUICK] FROM <tablename>  
[WHERE where\_clause] [ORDER BY ...] [LIMIT #]

mysql> **DELETE** FROM employee;

mysql> **DELETE** FROM employee WHERE dept\_code = 'MKTG';

mysql> **DELETE** quick from emp order by empno limit 2;

# Module 7. Modifying Table Structure

---

- Overview
  - Altering Table structure
  - Dropping Column from a Table
  - Dropping a Table

# Modifying a Table Structure

---

ALTER (IGNORE) TABLE <tablename> <alter phrase> [ , <alter phrase> ]

Common <alter phrase> variables to use with the ALTER TABLE statement include the following:

ADD <columnname> <create specification>

CHANGE (COLUMN) <oldcolumnname> <create specification>

MODIFY (COLUMN) <create specification>

DROP (COLUMN) <columnname>

RENAME (TO) <newtablename>

# Modifying a Table Structure

---

Mysql> **ALTER** table emp

**drop comm;**

Mysql> **ALTER** table emp

**Add email varchar(20);**

**or**

Mysql> **ALTER** table emp

**Add email varchar(20) first;**

**or**

Mysql> **ALTER** table emp

**Add email varchar(20) after sal;**

Mysql>**ALTER** table emp

**modify ename varchar(40);**

# Modifying a Table Structure

---

```
Mysql>alter table emp
```

```
change sal sal1 float(8,2);
```

```
Mysql>alter table emp
```

```
alter comm set default 1000;
```

```
Mysql>alter table emp
```

```
alter comm drop default;
```

```
Mysql>alter table emp
```

```
rename to newemp;
```

# Dropping a Table

---

```
DROP TABLE table-name;
```

```
MYSQL> DROP table dept;
```

```
MYSQL> DROP table IF EXISTS dept;
```

# Module 8. Integrity Constraints

---

- Overview

- Understanding Table and Column Constraints
- Creating, Modifying and Dropping Column level constraints
- Creating, Modifying and Dropping Table level constraints
- Adding Constraints to Columns of an existing table
- Enabling and Disabling Constraints
- Dropping Columns and Tables having constraints

# Integrity Constraints

---

- Not Null
- Unique
- Check
- Primary Key
- Foreign Key

# Column Constraints

---

```
mysql> CREATE TABLE employee (  
        emp_code number (5) NOT NULL,  
        emp_name varchar (25) NOT NULL,  
        dept_code varchar (4) );
```

# Column Constraints

---

```
mysql> CREATE TABLE employee (  
        emp_code int (5) UNIQUE,  
        emp_name varchar2 (25) NOT NULL);
```

# The UNIQUE Constraint

---

```
mysql> CREATE TABLE supplier (  
        supp_code int(4) PRIMARY KEY,  
        supp_name varchar (30) UNIQUE);
```

```
mysql > CREATE TABLE supplier (  
        supp_code int(4) PRIMARY KEY,  
        supp_name varchar (30) UNIQUE  
        NOT NULL);
```

# The CHECK Constraint

---

```
mysql> CREATE TABLE employee (  
        emp_code int(5) PRIMARY KEY,  
        emp_name varchar (25) NOT NULL,  
        dept_code varchar (4) CHECK (dept_code = upper  
        (dept_code) ));
```

# The PRIMARY KEY Constraint

---

```
mysql> CREATE TABLE employee (  
        emp_code int(5) PRIMARY KEY,  
        emp_name varchar (25) NOT NULL,  
        dept_code varchar (4));
```

# The REFERENCES Constraint

---

```
mysql> CREATE TABLE employee (  
        emp_code int(5)  
        PRIMARY KEY,  
        emp_name varchar (25) NOT NULL,  
        dept_code varchar (4)  
REFERENCES dept (dept_code));
```

# The REFERENCES Constraint

---

```
mysql> CREATE TABLE employee (  
    emp_code int(5) primary key,  
    emp_name varchar (25) not null,  
    dept_code varchar (4)  
    REFERENCES dept(dept_code)  
    ON DELETE CASCADE | ON DELETE SET NULL  
    ON UPDATE RESTRICT);
```

# Table Constraints

---

```
mysql> CREATE TABLE employee(  
    emp_code int(4)not null,  
    emp_name varchar(40)not null,  
    dept_code varchar(4),  
    UNIQUE (emp_code,emp_name) );
```

# The PRIMARY KEY and CHECK Constraint

---

```
mysql> CREATE TABLE orders (  
        order_year int (4),  
        order_number int(5),  
        order_date date,  
        PRIMARY KEY (order_year, order_number));
```

```
mysql> CREATE TABLE employee (emp_code int(4),  
        emp_name varchar(20),  
        date_birth date,date_join date,  
        CHECK (date_join > date_birth));
```

# The FOREIGN KEY Constraint

---

```
mysql> CREATE TABLE ship (  
    ship_code varchar (5),  
    ship_name varchar (20) not null,  
    PRIMARY KEY (ship_code) );
```

```
mysql> CREATE TABLE voyage (  
    ship_code varchar (5),  
    voyage_number int(3),  
    date_arrival date,  
    PRIMARY KEY  
    (ship_code,voyage_number),  
    FOREIGN KEY (ship_code)  
    REFERENCES ship (ship_code) );
```

# The FOREIGN KEY Constraint

---

```
mysql> CREATE TABLE docket (  
    docket_number int(5),  
    docket_date date,  
    ship_code varchar (5),  
    voyage_number int (3),  
    PRIMARY KEY (docket_number),  
    FOREIGN KEY (ship_code, voyage_number)  
    REFERENCES voyage (ship_code, voyage_number)  
    ON DELETE CASCADE);
```

# Removing Duplicates, Table Replacement:

---

```
mysql> CREATE TABLE tmp SELECT last_name, first_name, sex  
      FROM person  
      GROUP BY last_name, first_name;
```

```
mysql> DROP TABLE person;
```

```
mysql> ALTER TABLE tmp RENAME TO person;
```

Or

```
ALTER IGNORE TABLE person  
  ADD PRIMARY KEY (last_name, first_name);
```

```
Mysql>ALTER TABLE emp  
  ADD unique KEY (ename);
```

```
mysql>alter table emp  
      modify comm decimal(7,2) not null default 1000;
```

# Renumbering an Existing Sequence

---

```
Mysql>alter table emp
```

```
drop empno;
```

```
mysql>alter table emp
```

```
add empno int not null auto_increment first,
```

```
add primary key (empno);
```

# Module 9. Built-In Functions

---

- Overview

- String functions
- Numeric functions
- Date functions
- Special formats with Date data types
- Conversion functions

# String functions

| Function                      | Returns                                                 | Example                                                | Result                |
|-------------------------------|---------------------------------------------------------|--------------------------------------------------------|-----------------------|
| lcase (x)                     | Converts the entire string to lowercase.                | SELECT lcase<br>( 'Inder Kumar Gujral' )<br>FROM dual; | inder kumar<br>gujral |
| ucase(x)                      | Converts the entire string to uppercase.                | SELECT ucase<br>( 'Inder' ) FROM dual;                 | INDER                 |
| ascii(c)                      | ASCII value of the given character, C                   | SELECT ascii('ARTI');                                  | 65                    |
| Concat(s1,s2..<br>)           | Concatenates (joins) the strings, S1, S2                | SELECT concat('bina<br>, 'gore');                      | Bina gore             |
| replace (char,<br>str1, str2) | Every occurrence of str1 in char is replaced with str2. | SELECT replace( 'Cap' , 'C',<br>'M' );                 | Map                   |
| length (char)                 | Length of char.                                         | SELECT length ('MySQL');                               | 5                     |
| rtrim(str)                    | Trims whitespace from the right of string               | SELECT rtrim(' priti');                                | priti                 |

# String functions

| Function                    | Returns                                                                                                                    | Example                                      | Result    |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|-----------|
| Substr()                    | Return the substring as specified                                                                                          | SELECT<br>SUBSTRING('Quadratically',5)<br>;  | ra        |
| CHAR_LENGTH(str)            | Returns the length of the string str                                                                                       | SELECT<br>CHAR_LENGTH("Pragati");            | 7         |
| INSERT(str,pos, len,newstr) | Returns the string str, with the substring beginning at position pos and len characters long replaced by the string newstr | SELECT<br>INSERT('Quadratic', 3, 4, 'What'); | QuWhattic |
| INSTR(str,substr)           | Returns the position of the first occurrence of substring substr in string str.                                            | SELECT<br>INSTR('foobarbar', 'bar');         | 4         |
| LENGTH(str)                 | Returns the length of the string str                                                                                       | SELECT LENGTH('priti');                      | 5         |
| LPAD(str,len,padstr)        | Returns the string str, left-padded with the string padstr to a length of len characters.                                  | SELECT LPAD('hi',4,'??');                    | ??hi      |

# Numeric functions

| Function    | Returns                                             | Example                                                 | Result      |
|-------------|-----------------------------------------------------|---------------------------------------------------------|-------------|
| SQRT (n)    | square root of any number                           | select SQRT(16);                                        | 4           |
| RAND ()     | random numbers between 0 and 1                      | SELECT RAND();                                          |             |
| ABS(N)      | absolute value                                      | Select abs(-5);                                         | 5           |
| Sign(n)     | Return 1 or -1 or 0                                 | Select sign(-5);                                        | -1          |
| POW(n1,n2)  | Power of a number                                   | Select POW(5,3);                                        | 125         |
| Round(n,d)  | Round the number                                    | Select round(4.789,1);                                  | 4.8         |
| Format(x,d) | rounded to <i>D</i> decimal places                  | Select<br>format(12332.123456, 4);                      | 12,332.1235 |
| IFNULL()    | replace the null value column with a specific value | select empno, comm,<br>ifnull (comm, 1200) from<br>emp; |             |

# Numeric functions

| Function                 | Returns                                                                             | Example                                        | Result             |
|--------------------------|-------------------------------------------------------------------------------------|------------------------------------------------|--------------------|
| CEILING(X)               | return the smallest integer value that is not smaller than X.                       | SELECT CEILING(3.46);                          | 4                  |
| FLOOR(X)                 | returns the largest integer value that is not greater than X.                       | SELECT FLOOR(7.55);                            | 7                  |
| FORMAT(X, D)             | to format the number X in the following given format.                               | SELECT<br>FORMAT(423423234.65434453,2);        | 423,423,234<br>.65 |
| GREATEST(n1,n2,n3,.....) | returns the greatest value in the set of input parameters (n1, n2, n3, a nd so on). | SELECT<br>GREATEST(3,5,1,8,33,99,34,55,67,43); | 99                 |
| LEAST(N1,N2,N3,N4,.....) | to return the least-valued item from the value list                                 | SELECT<br>LEAST(3,5,1,8,33,99,34,55,67,43);    | 1                  |
| LOG10(X)                 | returns the base-10 logarithm of X                                                  | SELECT LOG10(100);                             | 2.000000           |

# Date and Time Functions

---

| Function                 | Returns                                           | Example                                                        | Result                     |
|--------------------------|---------------------------------------------------|----------------------------------------------------------------|----------------------------|
| ADDDATE()                | Add dates                                         | SELECT ADDDATE('1998-01-02',31);                               | 1998-02-02                 |
| ADDTIME(expr1,expr2)     | adds expr2 to expr1 and returns the result        | SELECT ADDTIME('1997-12-31 23:59:59.999999','1 1:1:1.000002'); | 1998-01-02 01:01:01.000001 |
| CURDATE()                | current date as a value in 'YYYY-MM-DD'           | SELECT CURDATE();                                              | 2013-03-30                 |
| CURTIME()                | Returns the current time as a value in 'HH:MM:SS' | SELECT CURTIME();                                              | 23:50:26                   |
| DATE(expr)               | Extracts the date part of the date or datetime    | SELECT DATE('2003-12-31 01:02:03');                            | 2003-12-31                 |
| DATEDIFF(expr1,expr2)    | Difference in days                                | SELECT DATEDIFF('1997-12-31 23:59:59','1997-12-30')            | 1                          |
| DATE_FORMAT(date,format) | Formats the date value                            | SELECT DATE_FORMAT('1997-10-04 22:23:00', '%W %M %Y');         | Saturday<br>October 1997   |

# Format specifier characters

---

| Specifier | Description                                                  |
|-----------|--------------------------------------------------------------|
| %a        | Abbreviated weekday name (Sun..Sat)                          |
| %b        | Abbreviated month name (Jan..Dec)                            |
| %c        | Month, numeric (0..12)                                       |
| %D        | Day of the month with English suffix (0th, 1st, 2nd, 3rd, .) |
| %d        | Day of the month, numeric (00..31)                           |
| %f        | Microseconds (000000..999999)                                |
| %H        | Hour (00..23)                                                |
| %h        | Hour (01..12)                                                |

# Format specifier characters

---

| Specifier | Description                                              |
|-----------|----------------------------------------------------------|
| %i        | Minutes, numeric (00..59)                                |
| %j        | Day of year (001..366)                                   |
| %l        | Hour (1..12)                                             |
| %M        | Month name (January..December)                           |
| %m        | Month, numeric (00..12)                                  |
| %p        | AM or PM                                                 |
| %r        | Time, 12-hour (hh:mm:ss followed by AM or PM)            |
| %S        | Seconds (00..59)                                         |
| %s        | Seconds (00..59)                                         |
| %T        | Time, 24-hour (hh:mm:ss)                                 |
| %U        | Week (00..53), where Sunday is the first day of the week |
| %u        | Week (00..53), where Monday is the first day of the week |

# Format specifier characters

---

| Specifier | Description                                                                                      |
|-----------|--------------------------------------------------------------------------------------------------|
| %W        | Weekday name (Sunday..Saturday)                                                                  |
| %w        | Day of the week (0=Sunday..6=Saturday)                                                           |
| %X        | Year for the week where Sunday is the first day of the week, numeric, four digits; used with %V  |
| %x        | Year for the week, where Monday is the first day of the week, numeric, four digits; used with %v |
| %Y        | Year, numeric, four digits                                                                       |
| %y        | Year, numeric (two digits)                                                                       |

# Date Functions

| Function                | Returns                                                            | Example                                 | Result     |
|-------------------------|--------------------------------------------------------------------|-----------------------------------------|------------|
| DAYNAME(date)           | name of the weekday                                                | SELECT DAYNAME('2013-03-30');           | SATURDAY   |
| DAYOFMONTH(date)        | day of the month for date, in the range 0 to 31.                   | SELECT DAYOFMONTH('1998-02-03');        | 3          |
| DAYOFWEEK(date)         | weekday index for date (1 = Sunday, 2 = Monday, .., 7 = Saturday). | SELECT DAYOFWEEK('1998-02-03');         | 3          |
| DAYOFYEAR(date)         | the day of the year for date, in the range 1 to 366.               | SELECT DAYOFYEAR('1998-02-03');         | 34         |
| EXTRACT(unit FROM date) | extracts parts from the date                                       | SELECT EXTRACT(YEAR FROM '1999-07-02'); | 1999       |
| HOUR(time)              | the hour for time. The range of the return value is 0 to 23        | SELECT HOUR('10:05:03');                | 10         |
| LAST_DAY(date)          | value for the last day of the month.                               | SELECT LAST_DAY('2003-02-05');          | 2003-02-28 |

# Date Functions

| Function              | Returns                                                       | Example                                                                     | Result                     |
|-----------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------|----------------------------|
| MONTH(date)           | the month for date, in the range 0 to 12                      | SELECT MONTH('1998-02-03')                                                  | 2                          |
| MONTHNAME(date)       | the full name of the month for date.                          | SELECT MONTHNAME('1998-02-05');                                             | February                   |
| NOW()                 | Returns the current date and time                             | SELECT NOW();                                                               | 1997-12-15<br>23:50:26<br> |
| SYSDATE()             | the current date and time as a value in 'YYYY-MM-DD HH:MM:SS' | SELECT SYSDATE();                                                           | 2006-04-12<br>13:47:44     |
| TIME(expr)            | Extracts the time                                             | SELECT TIME('2003-12-31 01:02:03');                                         | 01:02:03                   |
| TIMEDIFF(expr1,expr2) | Difference in time                                            | SELECT TIMEDIFF('1997-12-31 23:59:59.000001','1997-12-30 01:01:01.000002'); | 46:58:57.99<br>9999        |
|                       |                                                               |                                                                             |                            |

# Date Functions

---

| Function           | Returns                                                          | Example                                | Result |
|--------------------|------------------------------------------------------------------|----------------------------------------|--------|
| WEEK(date[, mode]) | This function returns the week number for date                   | SELECT WEEK('1998-02-20');             | 7      |
| WEEKDAY(date)      | weekday index for date (0 = Monday, 1 = Tuesday, . 6 = Sunday).  | SELECT WEEKDAY('1998-02-03 22:23:00'); | 1      |
| WEEKOFYEAR(date)   | calendar week of the date as a number in the range from 1 to 53. | SELECT WEEKOFYEAR('1998-02-20');       | 8      |
| YEAR(date)         | year for date, in the range 1000 to 9999                         | SELECT YEAR('98-02-03');               | 1998   |

# Conversion Functions

---

- The conversion functions are:
  - `str_to_date()`
  - `Binary()`
  - `Cast()`
  - `Convert()`

# Module 10. Indexes

---

- Overview
  - Understanding Indexes
  - Unique , Simple Indexes, Partial index
  - Creating and dropping Indexes
  - Querying the STATISTICS table

# Indexes

---

- Are database objects used to improve the performance of the database.
- Uses the ROWID for search operations.
- There are two types of index:
  - UNIQUE
  - SIMPLE

```
CREATE [UNIQUE] INDEX index_name ON table_name  
(column_name, column_name );
```

```
mysql> CREATE UNIQUE INDEX idx_dept_code ON  
employee(dept_code);
```

# Simple Indexes

---

Mysql> **CREATE INDEX** idx\_dept\_code **ON** employee (dept\_code);

Mysql> **CREATE INDEX** emp\_idx **ON** employee (dept\_code, emp\_code);

mysql> **DROP INDEX** name\_index on table\_name;

# Partial index

---

```
Mysql> create index name_idx1  
on person (name(7));
```

or

```
Mysql> alter table person  
add key name_idx (name(7));
```

# ALTER command to add INDEX:

---

```
mysql> alter table emp  
        add primary key (eno);  
mysql> alter table dept  
        add unique u_dname (dname);  
mysql> alter table emp  
        add index ind_job(job);
```

Dropping the index

```
mysql > DROP INDEX name_index on table_name;  
Mysql> ALTER TABLE emp DROP PRIMARY KEY;
```

# Querying the Data Dictionary

---

```
mysql> show index from emp;
```

- Mysql> SELECT DISTINCT TABLE\_NAME,  
INDEX\_NAME FROM  
INFORMATION\_SCHEMA.STATISTICS WHERE  
TABLE\_SCHEMA = 'pragati'

# Module 11. Advanced Queries

---

- Overview
  - Table joins
  - Sub queries
  - Set operators
  - MERGE statement

# JOINS

---

## Cross Join

```
mysql> SELECT emp_name, dept_name  
        FROM EMPLOYEE  
        CROSS JOIN DEPT
```

# JOINS

---

## Inner Join

```
mysql> SELECT emp_code, dept_name  
        FROM employee a JOIN dept b  
        ON (a.dept_code=b.dept_code)
```

# JOINS

---

## Equi Join

```
mysql> SELECT emp_name, dept_name  
        FROM EMPLOYEE, DEPT  
        WHERE dept.dept_code = employee.dept_code;
```

# JOINS

---

## Left Outer Join

```
mysql> SELECT A.emp_name, B.emp_name  
        FROM employee A  
        left join dept B  
        on A.reports_to = B.emp_code ;
```

# JOINS

---

## Right Outer Join

```
Mysql>SELECT A.emp_name, B.emp_name  
        FROM employee A  
        right join dept B  
        on A.reports_to = B.emp_code ;
```

# JOINS

---

```
mysql> SELECT e.ename, d.dname  
        FROM emp e  
        left join dept d using (deptno)
```

```
mysql> SELECT emp_code, dept_name  
        FROM employee a JOIN dept b  
        ON (a.dept_code=b.dept_code  
        And a.emp_code<20);
```

# JOINS

---

## Self Join

```
mysql> SELECT a.emp_name, b.emp_name  
        FROM employee A, employee B  
        WHERE A.reports_to = B.emp_code;
```

# SUBQUERIES

---

```
mysql> SELECT * FROM emp  
      WHERE deptno =(SELECT  FROM dept  
      WHERE dname = 'operations');
```

```
mysql> SELECT * FROM orders  
      WHERE cust_code IN (SELECT  cust_code FROM customer  
      WHERE city_code = 'PUNE');
```

```
mysql > SELECT * FROM dept WHERE EXISTS (  
      SELECT * FROM employee WHERE employee.dept_code =  
      dept.dept_code);
```

```
mysql > SELECT * FROM dept WHERE NOT EXISTS (  
      SELECT * FROM employee WHERE employee.dept_code =  
      dept.dept_code);
```

# SUBQUERIES

---

➤Checking the existence of the inner query.

```
Mysql>select deptno from emp where EXISTS (select deptno  
from dept);
```

```
mysql> SELECT * FROM employee WHERE salary = (SELECT MIN  
      (salary) FROM employee);
```

# SET Operators

---

```
mysql> SELECT prod_code, prod_name FROM product
      UNION
      SELECT prod_code, prod_name FROM old_products;
```

```
Mysql>SELECT prod_code, prod_name FROM product
      UNION distinct
      SELECT prod_code, prod_name FROM old_product;
```

```
Mysql>SELECT prod_code, prod_name FROM product
      UNION all
      SELECT prod_code, prod_name FROM old_product;
```

# SET Operators

---

The result wise UNION, UNION ALL, DISTINCT is same.

Use UNION ALL instead of simple UNION, and DISTINCT the output will be same but for the optimization of the query can be increased. It is 3.5 times faster.

# MERGE statement

---

- Meaning
  - Mysql Merge Statement is used to merge the two sql statement using UNION clause..
  - The UNION clause is used to combine the result set of any two sql queries.

# MERGE statement

---

- Example

➤ Mysql>create table e\_merge(  
empno decimal(4,0),  
ename varchar(10) ) engine=merge union=(e\_t1,e\_t2);

# Module 12. Views

---

- Overview
  - Understanding Views
  - Creating views
  - Altering & dropping views
  - Manipulating data using views
  - Viewing the Details

# Views

---

- View is a virtual table
- Doesn't store any data
- Used to extract the data from multiple tables

## Restrictions on View Definitions

- Can't have triggers on views
- cannot contain a subquery in the FROM clause of the SQL statement.

# Views

---

## Restrictions on View Definitions

- User, system, or local variables are not allowed in the SQL SELECT statement.
- Views can't point at temporary tables.
- Views created within stored procedures can't reference parameters in the stored procedure.

# Views

---

```
CREATE [OR REPLACE] [<algorithm attributes>] VIEW  
[database.]< name> [<columns>]  
AS <SELECT statement> [ WITH CHECK OPTION ]
```

# Views

---

```
mysql> CREATE VIEW fin_emp AS  
      SELECT * FROM employee WHERE dept_code = 'FIN';
```

```
mysql > SELECT * FROM fin_emp;
```

```
Mysql> CREATE VIEW emp_view(emp_name,employee_no)  
as select ename, empno from emp where deptno=10
```

With check option;

```
mysql> DELETE from fin_emp;
```

# Views

---

```
mysql> insert into fin_emp (emp_code, emp_name, dept_code)  
      values (111, 'Sunil', 'FIN');
```

```
mysql> CREATE OR REPLACE VIEW fin_emp AS  
      SELECT * FROM employee;
```

# Views

---

```
mysql> DROP VIEW fin_emp;
```

➤ Restriction:

We can not use order by clause

# Viewing the Details

---

```
mysql> show create view fin_emp \g;
```

```
mysql> explain select * from fin_emp;
```

# Module 13. Transaction Processing

---

- Overview
  - ACID
  - Transaction Processing commands
  - Locking table

# Properties of Transactions(ACID):

---

- Atomicity
- Consistency
- Isolation
- Durability

# Transaction Processing

---

- COMMIT
- ROLLBACK

# Commit

---

- COMMIT is used to write the changes to the database.
- MySQL automatically commit the changes to the database.
- To force MySQL not to commit the changes automatically following statement is used:
  - Set autocommit = 0;

# Transaction Processing

---

mysql> SAVEPOINT savepointname;

*mysql*> SAVEPOINT stage1;

mysql> ROLLBACK TO savepointname;

mysql> ROLLBACK TO stage1;

# Transaction Processing

---

lock table table\_reference\_list lock\_type ;

mysql> LOCK TABLES *gadgets* [[AS] *alias*] read;

Mysql> UNLOCK TABLES;

Lock\_type:  
write or read

# Module 14. Miscellaneous function

---

- Overview
  - `if()`
  - `isnull()`
  - `nullif()`
  - `case`
  - Loading the data from text file

# Logical and Conditional Functions

---

## **If() function**

```
mysql>select sal, if (sal> 1200, 1,0) from emp;
```

# Logical and Conditional Functions

---

## **IsNull() function**

To determine whether the value of the argument given in parentheses is NULL.

### **It returns**

- 1 if the value is NULL
- 0 if it is not NULL.

```
mysql> select ename, comm from emp  
where isnull(comm);
```

# Logical and Conditional Functions

---

## **null()if function**

This MySQL function returns NULL if the two arguments given are equal. Otherwise, it returns the value or results of the first argument.

### **Syntax**

NULLIF(*condition1*, *condition2*)

|                           |   |      |
|---------------------------|---|------|
| select nullif(null,null); | ⇒ | null |
| select nullif(500,null);  | ⇒ | 500  |
| select nullif(500,300);   | ⇒ | 500  |

# Cases

---

```
mysql> SELECT ename,  
      (  
        case deptno  
        when 10 then 'ACCOUNTS'  
        when 20 then 'RESEARCH'  
        when 30 then 'SALES'  
        when 40 then 'OPERATIONS'  
        else 'UNASSIGNED'  
        end  
      ) as Department  
FROM emp;
```

# Loading the data from text file

---

- Create a text file eg: c:\sal.txt
- Command:

Mysql>Load data local infile 'c:/sal.txt' into table salary;

For inserting the null values,

|       |     |      |
|-------|-----|------|
| Priti | Ght | 7000 |
| Bina  | \N  | 9000 |
| Rita  | \N  | 8000 |

# Module 15. Introduction to programming

---

- Overview
  - creating local variables
  - storing the data from the table
  - Looping and Conditional constructs

# User-Defined variables

---

```
Mysql>set @name='Anuradha';
```

```
Mysql>set @tel_no=5432487;
```

```
Mysql>select @name, @tel_no;
```

# User-Defined variables

---

```
Mysql>select @max_sal:=max(sal)
```

```
From emp
```

```
Mysql>select * from emp
```

```
Where sal=@max_sal;
```

# Compound-Statement

---

- used for writing compound statements
- can contain multiple statements
- can appear within stored programs (stored procedures and functions, and triggers).

```
begin
    update employee
    set salary = salary + 111
    where dept_code = 'MKTG';
    delete from employee
    where dept_code = 'operation';
end;
```

# Example

---

```
Mysql>Create procedure GetProducts()  
BEGIN  
Select * from Products  
END ;
```

# Repeat loop

---

```
delimiter //
CREATE PROCEDURE dorepeat1(p1 INT)
BEGIN
    SET @x = 0;
    REPEAT
        SET @x = @x + 1;
        select @x;
    UNTIL @x > p1
    END REPEAT;
end
//
delimiter ;
call dorepeat1(20);
```

# IF-ELSE Statements

---

```
if nValue > 40 then  
    set nCount = nCount + 1;  
end if;
```

```
if nValue > 40 then  
    if nValue < 50 then  
        set nCount = nCount + 1;  
    end if;  
end if;
```

```
if nValue between 40 and 50 then  
    set nCount = nCount + 1;  
end if;
```

# IF-ELSE Statements

---

```
if nValue > 40 then
    set nCount1 = nCount1 + 1;
Else
    set nCount2 = nCount2 + 1;
end if;
```

```
if nSalary < 2000 then
    set nProfTax = 0;
Elseif nSalary < 3500 then
    set nProfTax = 15;
elseif nSalary < 5000 then
    set nProfTax = 30;
else
    set nProfTax := 50;
end if;
```

# Example

---

```
Mysql>create function total (a int, b int) returns varchar(50)
begin
    declare c int;
    declare s varchar(50);
    set c = a + b;
    if c < 10 THEN
        set s = 'add is less then 10';
    ELSE
        set s = 'add is equal or greater then 10';
    END IF;
    RETURN s;
END $$
```



# Example else-if

---

```
Mysql>create function check1(a int, b int) returns varchar(50)
begin
declare c int;
declare s varchar(50);
if a < b THEN
set s = 'a is less than b';
ELSEif a > b THEN
set s ='add is greater than b';
else
set s='a is equal to b';
END IF;
RETURN s;
END $$
```



# Example else-if

---

➤ To execute function

```
DELIMITER ;  
select check1 (10, 20);
```



# Example loop ... end loop

---

```
Mysql> create procedure proce2()
BEGIN
    DECLARE count INT default 0;
    DECLARE in_count int;
    set in_count =20;
    increment: LOOP
        SET count = count + 1;
        select count;
        IF count < 20 THEN ITERATE increment; END IF;
        IF count > in_count THEN LEAVE increment;
    END IF;
    END LOOP increment;
SELECT count;
END
```



# Module 16. Temporary tables

---

- Overview
  - Creating temporary tables
  - Table Cloning
  - Getting Server Metadata

# Usage of temporary table

---

```
mysql >CREATE TEMPORARY TABLE SalesSummary (  
    product_name VARCHAR(50) NOT NULL,  
    total_sales DECIMAL(12,2) NOT NULL DEFAULT 0.00,  
    avg_unit_price DECIMAL(7,2) NOT NULL DEFAULT 0.00,  
    total_units_sold INT UNSIGNED NOT NULL DEFAULT 0  
);
```

# Dropping Temporary Tables:

---

```
Mysql>DROP TABLE SalesSummary;
```

# MySQL Clone Tables

---

## Step 1

```
mysql > SHOW CREATE TABLE emp \G;
```

## Step 2

Rename this table and create another table

```
mysql > CREATE TABLE clone_EMP  
(  
    EMPNO NUMERIC(4) not null,  
    ENAME VARCHAR(10),  
    JOB VARCHAR(9),  
    MGR NUMERIC(4),  
    HIREDATE DATE,  
    SAL NUMERIC(7,2),  
    COMM NUMERIC(7,2),  
    DEPTNO NUMERIC(2) REFERENCES DEPT(DEPTNO)  
) ENGINE =InnoDB;
```

# MySQL Clone Tables

---

## Step 3

```
Mysql>insert into  
    clone_emp1(empno,ename,job,mgr,hiredate,sal,  
    comm, deptno)  
select empno,ename,job,mgr,hiredate,sal,  
    comm, deptno from emp;
```

# Getting Server Metadata( in MySQL)

---

| Command                   | Description                                      |
|---------------------------|--------------------------------------------------|
| <b>SELECT VERSION( )</b>  | <b>Server version string</b>                     |
| <b>SELECT DATABASE( )</b> | <b>Current database name<br/>(empty if none)</b> |
| <b>SELECT USER( )</b>     | <b>Current username</b>                          |
| <b>SHOW STATUS</b>        | <b>Server status indicators</b>                  |
| <b>SHOW VARIABLES</b>     | <b>Server configuration<br/>variables</b>        |

# Module 17. Cursors

---

- Overview
  - Understanding Cursors
  - Types of cursors
  - Cursor operations

# Cursors

---

- Allow you to manipulate a row -by-row processing of the resultsets.
- The cursor allows you to iterate through the result set
- Can perform certain operations on each row.
- A cursor enables a programmer or DBA to perform complex processing on the server within the database.
- Operate on all the rows in the returned by a query at one time.
- Cursor is convenient to use when you are performing on a complex resultset

# Cursors

---

- Main purpose of a cursor is when you want to perform operation in **multiple tables** for each row with the results of a query operations.
- Another reason to use cursor is to use when there is some steps in process are optional and you want to perform those steps on certain rows of query.
- so with cursor you can fetch the result set and then perform the additional processing only on the rows that require it.

# Types of Cursors

---

1. **Asensitive**

2. **Read Only**

3. **Non\_Scrollable** (or **forward-only**)

- `employee_cur CURSOR For select * from employee;`
- `OPEN employee_cur;`
- `FETCH employee_cur INTO var_name;`
- `CLOSE employee_cur;`

# Types of Cursors

---

1. **Asensitive**
2. **Read Only**
3. **Non\_Scrollable** (or **forward-only**)

# Steps of Cursors

---

1. Declare a cursor
2. Open a cursor statement
3. Fetch the cursor
4. Close the cursor
5. Handler

# Steps of Cursors

---

## 1. Declare..

- To declare a cursor

## 2. Open a cursor statement

- This process actually retrieves the data using the previously defined SELECT statement. With the cursor populated with data, individual rows can be fetched

## 3. Fetch the cursor

- When we have to retrieve the next row from the cursor

## 4. Close the cursor

- CLOSE frees up any internal memory and resources used

# Disadvantage of Cursors

---

- The major disadvantage to using a cursor is the performance hit.
- Cursors take quite a few resources to accomplish their tasks.

# Cursors

---

```
Mysql> declare
        sales cursor for
        SELECT
        a.salesman_id,a.salesman_name, b.target, c.sales
        FROM salesman a, target b, sales c
        WHERE a.salesman_id=b.salesman_id
        and b.tmonth=c.smonth;

begin
        .....
end;
```

# Cursors

---

```
mysql>CREATE PROCEDURE curdemo()  
BEGIN  
  DECLARE done INT DEFAULT FALSE;  
  DECLARE v_roll int(11) DEFAULT 0;  
  DECLARE t1_cur CURSOR FOR SELECT * from t1;  
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;  
  
  OPEN t1_cur;  
  read_loop: LOOP  
    FETCH t1_cur INTO v_roll;  
    IF done THEN  
      LEAVE read_loop;  
    END IF;  
    select concat('the roll number is ', v_roll);  
  END LOOP;  
  CLOSE t1_cur;  
END;
```

# Cursors

---

## DECLARE ... HANDLER Syntax

DECLARE *handler\_action* HANDLER FOR *condition\_value* [, *condition\_value*]  
... *Statement*

- The **DECLARE ... HANDLER** statement specifies a handler that deals with one or more conditions.
- If one of these conditions occurs, the specified *statement* executes.
- *statement* can be a simple statement such as SET *var\_name* = *value*, or a compound statement written using BEGIN and END
- Handler declarations must appear after variable or condition declarations.

# Cursors

---

- Disable DATA NOT FOUND handlers from calling functions

```
DECLARE CONTINUE HANDLER FOR SQLSTATE '02000' BEGIN END;
```

```
DECLARE CONTINUE HANDLER FOR SQLSTATE '02000' SET done=1;
```

- Setting the value

```
DECLARE CONTINUE HANDLER FOR NOT FOUND SET  
flag=1;
```

# Cursors

---

```
Mysql>CREATE PROCEDURE curdemo()  
BEGIN  
  DECLARE done INT DEFAULT FALSE;  
  DECLARE v_deptno decimal(2,0);  
  DECLARE v_dname varchar(14);  
  DECLARE v_loc varchar(13);  
  DECLARE cur1 CURSOR FOR SELECT * from dept;  
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;  
  OPEN cur1;  
  read_loop: LOOP  
    FETCH cur1 INTO v_deptno,v_dname,v_loc;  
    IF done THEN  
      LEAVE read_loop;  
    END IF;  
    INSERT INTO dept1 VALUES (v_deptno,v_dname,v_loc);  
  END LOOP;  
  CLOSE cur1;  
END;
```

# Cursors

---

## **Advantages of cursor:**

- Best used when you want each row or more than one row one by one.
- Its efficient because with Cursor we are doing operations so there is no need to write complex queries(like joins)

## **Disadvantages:**

- Cursor is faster than a while loop but it create more overhead in database.
- Cursor fetching one by one data from database so if data is more, its take more execution time.

# Module 18. Stored Procedure and Functions

---

- Overview
  - Understanding Stored Procedures
  - Creating Stored Procedures
  - Parameter modes
  - Understanding Stored Functions
  - Creating Stored Functions
  - Transaction
  - Function

# Stored Procedures

---

- Stored Procedures are much faster, reusable
- It avoids the overhead of network communication
- Avoid parsing, optimizing

# Stored Procedures

---

```
CREATE [DEFINER = { user | CURRENT_USER }]
PROCEDURE sp_name ([proc_parameter[,...]])
[characteristic ...] routine_body
```

# Stored procedure

---

- Since MySQL version 5.0 stored procedure feature have been added to MySQL.
- For eg:  
    Create procedure GetProducts()  
    BEGIN  
    Select \* from Products  
    END ;
- Calling stored procedure  
    Call GetProducts();

# Stored Procedures

---

- Procedure parameter modes
  - The **IN** parameter mode is used to pass values to the procedure when invoked.
  - The **OUT** parameter mode is used to return a values to the caller of the procedure.
  - The **INOUT** parameter is used to pass initial values to the procedure when invoked and it also returns updated values to the caller.

# Stored Procedure

---

- Creating a Single-Statement Procedure

```
mysql> create procedure myProc ()  
SELECT id,first_name FROM employee;
```

```
mysql> call myProc ();
```

```
mysql> drop procedure myProc;
```

# Stored Procedure

---

Creating procedure with multiple statements:

```
Mysql>delimiter //
```

```
Mysql>CREATE procedure myProc ()  
BEGIN  
SELECT ename, sal FROM emp;  
SELECT deptno, dname FROM dept;  
END;  
//
```

```
Mysql>delimiter ;  
Mysql>CALL myProc ();
```

# Stored Procedure : in parameter

---

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE myProc (IN in_count INT)
BEGIN
DECLARE count INT default 0;
increment: LOOP
SET count = count + 1;
IF count < 20 THEN ITERATE increment; END IF;
IF count > in_count THEN LEAVE increment;
END IF;
END LOOP increment;
SELECT count;
END
//
mysql> DELIMITER ;
mysql> call myProc(5);
```

# Stored Procedure : out parameter

---

```
mysql> delimiter //
mysql>
mysql> CREATE PROCEDURE procout (OUT p_tot_sal
INT)
BEGIN
SELECT sum(sal) INTO p_tot_sal FROM emp;
END;
//
mysql> delimiter ;
mysql>
mysql> CALL procout(@sal);
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT @sal;
```

# Procedure

---

Creating procedure:

```
CREATE PROCEDURE simpleproc (OUT param1 INT)
BEGIN
SELECT COUNT(*) INTO param1 FROM emp;
END;
```

```
CALL simpleproc (@a);
select @a;
```

```
drop procedure simpleproc;
```

# Stored Procedure : in out parameter

---

```
mysql> delimiter //  
mysql> CREATE PROCEDURE update_sal (INOUT  
in_increment INT)  
BEGIN  
update emp  
set sal= sal+ in_increment where empno=700;  
SELECT sal INTO in_increment FROM emp  
where empno=700;  
END;  
//
```

```
mysql> delimiter ;  
Mysql>set @up_sal=1000;  
mysql> CALL update_sal(@up_sal);  
mysql> SELECT @up_sal;
```

# Stored Procedure :

---

To see information about Stored Procedure like:

- name of database
- type of procedure
- language
- code writtern
- date of creation

```
mysql>SELECT * FROM mysql.proc WHERE name  
= 'myProc'\G;
```

# Stored Procedure :

---

In case you would want to view all the stored procedures in a Database then we can use :

```
mysql>SELECT ROUTINE_NAME  
FROM INFORMATION_SCHEMA.ROUTINES  
WHERE ROUTINE_TYPE="PROCEDURE"  
AND ROUTINE_SCHEMA="dbname";
```

# Stored Functions

---

- Stored Functions
  - Is similar to stored procedures except that a function returns only a single values.

Syntax:

```
CREATE FUNCTION f_name ([parameter(s)])  
RETURNS data type  
DETERMINISTIC  
STATEMENTS
```

# Stored Functions

---

```
mysql>delimiter //  
mysql>create function hello (s char(20))  
returns char(50)  
deterministic  
return CONCAT('Hello',space(1),s);  
//
```

```
mysql>delimiter ;  
mysql>select hello('world');
```

# Stored Functions

---

```
Mysql>DELIMITER //  
Mysql>CREATE FUNCTION join_date_ck(return_date DATE)  
RETURNS VARCHAR(3)  
deterministic  
BEGIN  
DECLARE sf_value VARCHAR(3);  
IF curdate() > return_date  
THEN SET sf_value = 'Yes';  
ELSEIF curdate() <= return_date  
THEN SET sf_value = 'No';  
END IF;  
RETURN sf_value;  
END//
```

# Stored Functions

---

To run the function:

```
Mysql>DELIMITER ;
```

```
Mysql>select empno, hiredate, curdate(), join_date_ck(hiredate)  
from emp;
```

# Module 19. Triggers

---

- Overview
  - Understanding Triggers
  - Applying Triggers
  - Types of Triggers
  - Dropping Triggers

# Database Triggers

---

- A trigger defines an action the database should take when some database related event occurs.
- It may be used to supplement declarative referential integrity, to enforce complex business rules or to audit changes of data.
- Uses of Triggers are:
  - Audit Data Modifications
  - Log Events Transparently
  - Enforce Complex Business Rules
  - Derive Column Values Automatically
  - Implement Complex Security Authorizations
  - Maintain Replicate Tables

# Database Triggers

---

It has the following syntax:

```
CREATE TRIGGER <name> <time> <event>  
ON <table>  
FOR EACH ROW  
<body statements>
```

# Database Triggers

|             |                                                                                                                                                                                                                                                                              |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Triggername | Is the name of the trigger to be created                                                                                                                                                                                                                                     |
| BEFORE      | engine fires the trigger before executing the triggering statement.                                                                                                                                                                                                          |
| AFTER       | engine fires the trigger after executing the triggering statement.                                                                                                                                                                                                           |
| DELETE      | engine fires the trigger whenever a DELETE statement removes a row FROM the table.                                                                                                                                                                                           |
| INSERT      | Indicates that the engine fires the trigger whenever a INSERT statement adds a row to table.                                                                                                                                                                                 |
| UPDATE      | Indicates that the engine fires the trigger whenever an UPDATE statement changes a value in one of the columns specified in the OF clause. If the OF clause is omitted, the engine fires the trigger whenever a UPDATE statement changes a value in any column of the table. |

# Database Triggers

|              |                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ON           | Specifies the schema and name of the table, which the trigger is to be created. If schema is omitted, the engine assumes the table is in the user's own schema. <b>A trigger cannot be created on a table in the schema SYS.</b>                                                                                                                                                                                              |
| REFERENCING  | Specifies correlation names. Correlation names can be used in the block and WHEN clause of a row trigger to refer specifically to old and new values of the current row. The default correlation names are OLD and NEW. <b>If the row trigger is associated with a table named OLD or NEW, this clause can be used to specify different correlation names to avoid confusion between table name and the correlation name.</b> |
| FOR EACH ROW | Designates the trigger to be a row trigger. The engine fires a row trigger once for each row that is affected by the triggering statement and meets the optional trigger constraint defined in the when clause. If this clause is omitted the trigger is a statement trigger                                                                                                                                                  |
| WHEN         | Specifies the trigger restriction. The trigger restriction contains a SQL condition that must be satisfied for the engine to fire the trigger. This condition must contain correlation names and cannot contain a query. <b>Trigger restriction can be specified only for the row triggers.</b> The engine evaluates this condition for each row affected by the triggering statement.                                        |

# Database Triggers

---

MySQL triggers cannot:

- Use SHOW, LOAD DATA, LOAD TABLE, BACKUP DATABASE, RESTORE, FLUSH and RETURN statements.
- Use statements that commit or rollback implicitly or explicitly such as COMMIT, ROLLBACK, START TRANSACTION, LOCK/UNLOCK TABLES, ALTER, CREATE, DROP, RENAME... etc.
- Use prepared statement such as PREPARE, EXECUTE...
- Use dynamic SQL.
- Call a stored procedure or stored function.

# Database Triggers

---

- Applying Triggers
  - Triggering Event
    - It can be Insert, Update or Delete statement for a table
  - Trigger Constraint (Optional)
    - A boolean expression for each row trigger specified using a WHEN clause
  - Trigger Action
    - code to be executed when a triggering statement is encountered

# Database Triggers

---

- Types of Triggers

- The 'time' when the trigger fires

- BEFORE trigger (before the triggering action).
    - AFTER trigger (after the triggering action)

- The 'item' the trigger fires on

- Row trigger: once for each row affected by the triggering statement
    - Statement trigger: once for the triggering statement, regardless of the number rows affected

# Database Triggers

---

BEFORE triggers are used when the trigger action should determine whether or not the triggering statement should be allowed to complete.

By using a BEFORE trigger, you can eliminate unnecessary processing of the triggering statement.

BEFORE triggers are used to derive specific column values before completing a triggering INSERT or UPDATE statement.

# Database Triggers

---

- Expressions in Triggers
  - Help in referring to values in row triggers.
  - Need to use :OLD and :NEW prefixes.

If NEW.column\_name < OLD.column\_name.....

# Database Triggers

---

- Conditional Predicates
  - Useful when the trigger fires more than one type of DML operation.
  - Need to use the INSERTING, UPDATING or DELETING clause.
  - These are pre – defined PL/SQL Boolean type variables which evaluate to either true or false.

```
IF DELETING ('column_name') THEN.....
```

# Database Triggers

---

```
Mysql>delimiter $$  
Mysql>CREATE TRIGGER myTrigger  
BEFORE DELETE ON emp  
FOR EACH ROW  
BEGIN  
INSERT into transaction_log (user_id, description)  
VALUES (user(), 'Employee deleted ');  
END$$  
Mysql>delimiter ;
```

# Database Triggers

---

To execute the trigger:

```
Mysql>Delete from emp  
Where empno=7566;
```

To check ;

```
Mysql> select * from transaction_log
```

# Database Triggers

---

```
mysql> delimiter $$  
mysql> CREATE TRIGGER myTrigger  
BEFORE INSERT ON emp  
FOR EACH ROW  
BEGIN  
IF NEW.sal < 600 THEN  
SET NEW.ename='Y';  
ELSE  
SET NEW.ename='N';  
END IF;  
END$$
```

# Database Triggers

---

```
Mysql>Delimiter //
```

```
mysql> create trigger tr1
```

```
-> before insert on dept
```

```
-> for each row
```

```
-> begin
```

```
-> set @name='insert trigger ';
```

```
-> end;
```

```
-> //
```

# Database Triggers

---

To execute

```
Mysql>delimiter ;
```

```
Insert into dept values(50,'new','new');
```

```
Select @name;
```

# Dropping Triggers, View Details

---

- Dropping a trigger
  - delimiter //
  - drop trigger if exists trigger name ;
- View Details of triggers
  - `SELECT * FROM INFORMATION_SCHEMA.TRIGGERS;`

# Module 20. MySQL Prepared Statement

---

- Overview
  - Transactions
  - use of Prepared Statements
  - Benefits of Prepared Statements
  - Syntax

# Transactions

---

- MySQL support transactions.
- Storage engine type is used InnoDB.
- For eg:
  - delimiter \$\$
  - start transaction;
  - select @catid := max(categoryid) From categories;
  - Set @catid=@catid+1;
  - Insert into categories (categoryid, categoryname) values (@catid, 'Chocolates');
  - Insert into products (productname, unitprice, unitsinstock, categoryid) values ('Dairy Milk Silk', 50, 5, @catid);
  - Commit;
  - \$\$

# MySQL prepared statement

---

- By using Prepared statement server does not have to parse the query fully.
- It uses client/server binary protocol.
- It is faster when the query is executed multiple times.
- It uses placeholder to save statement.

# MySQL prepare statement

---

- MySQL 4.1 and newer support server-side *prepared statements*.
- client library can execute the query repeatedly by specifying the statement handle
- Prepared statements can have parameters
- Parameters can given as question-mark, called as placeholders
- You can repeat this as many times as desired.

# MySQL prepare statement

---

## ❑ Benefits of prepared statement

- more efficient than executing a query repeatedly
- The server has to parse the query only once, which saves some parsing
- The server has to perform some query optimization steps only once, as it caches a partial query execution plan
- Sending parameters via the binary protocol is more efficient than sending them as ASCII text.
- The binary protocol therefore helps save memory on the client,
- Also reduces network traffic

# MySQL prepare statement

---

## ❑ Benefits of prepared statement

- Only the parameters—not the entire query text—need to be sent for each execution, which reduces network traffic.
- Prepared statements can also help with security.
- A prepared statement is specific to the session

# MySQL prepare statement

---

- Prepare statement is based on other three MySQL statements as follow:
  - PREPARE :- to prepare statement for execution.
  - EXECUTE :- to execute a prepare statement
  - DEALLOCATE PREPARE :- to release a prepare statement.

# Prepare statement ...

---

```
mysql> SET @sql := 'SELECT actor_id, first_name, last_name  
-> FROM sakila.actor WHERE first_name = ?';  
mysql> PREPARE stmt_fetch_actor FROM @sql;  
mysql> SET @actor_name := 'Penelope';  
mysql> EXECUTE stmt_fetch_actor USING @actor_name;  
mysql> DEALLOCATE PREPARE stmt_fetch_actor;
```

# Module 21. Exception Handling

---

- Overview
  - Understanding Exception Handling
  - Using HANDLER
  - List of Exceptions

# Exception Handling

---

- Usually handled by try .. catch block in .NET environment
- Handled by using
  - DECLARE CONTINUE HANDLER
- Need to declare the handler
- should not use MySQL error code 0 or SQLSTATE values that begin with '00'

# Exception Handling

---

## Syntax

```
DECLARE handler_action HANDLER  
  FOR condition_value [, condition_value] ...  
  statement
```

*handler\_action*:

```
  CONTINUE  
| EXIT  
| UNDO
```

# Exception Handling

---

```
mysql> CREATE PROCEDURE handlerdemo ()  
BEGIN  
DECLARE CONTINUE HANDLER FOR SQLSTATE '23000' SET @x2 = 1;  
SET @x = 1;  
INSERT INTO test.t VALUES (1);  
SET @x = 2;  
INSERT INTO test.t VALUES (1);  
SET @x = 3;  
END;
```

```
mysql> CALL handlerdemo();//
```

```
mysql> SELECT @x//
```

# Exception Handling

---

| Error code | Message                                       |
|------------|-----------------------------------------------|
| 1240       | Key reference and table reference don't match |
| 1062       | Duplicate key in index                        |
| 1005       | Can't create table                            |
| 1006       | Can't create database                         |
| 1008       | Can't drop database                           |
| 1016       | Can't open file                               |
| 1036       | Table is read only                            |
| 1037       | Out of memory                                 |
| 1044       | Access denied for user                        |
| 1046       | No database selected                          |
| 1068       | Multiple primary key defined                  |